Stream Reverse Engineering

A Practical Step-by-Step Guide

How to find direct playable streams from any web source using pure Node.js. No browser needed. Covers movies, TV shows, anime, and all content types. Includes Cloudflare bypass, WASM decryption, XOR segments, custom ciphers, and building decryption proxies — explained simply.

What You Will Learn:

- How to identify what framework a streaming site uses
- How to find the hidden API that serves stream URLs
- How to bypass Cloudflare protection without a browser
- How to decrypt WASM, AES, and XOR encrypted streams
- How to build a local proxy that makes encrypted streams playable
- How to verify your streams work with ffprobe/ffmpeg

Table of Contents

1. The Big Picture 4

1.1 The Universal Chain . . . 4

1.2 What You Need . . . 4

2. Step 1: Reconnaissance (Scout the Site) 5

2.1 Fetch the Homepage . . . 5

2.2 Fetch a Movie Page . . . 5

2.3 Look for Clues . . . 5

3. Step 2: Find the Hidden API 6

3.1 Download JavaScript Bundles . . . 6

3.2 Search for API URLs . . . 6

3.3 Understanding Webpack Chunks . . . 6

4. Step 3: Bypass Cloudflare 7

4.1 Protection Type 1: TLS Fingerprinting (JA3) . . . 7

4.2 Protection Type 2: User-Agent Detection . . . 7

4.3 Protection Type 3: Referer Gating . . . 7

4.4 Quick Decision Tree . . . 7

5. Step 4: Decrypt Encrypted Streams 8

5.1 Base64-Encoded URLs (Easy) . . . 8

5.2 XOR-Encrypted Segments (Medium) . . . 8

5.3 WASM Decryption (Hard) . . . 8

5.4 AES-256-CBC Decryption (Hard) . . . 9

5.5 SHA-256 Obfuscated Field Names . . . 9

5.6 Custom Stream Ciphers (Hardest) . . . 9

6. Building a Decryption Proxy 11

6.1 How the Proxy Works . . . 11

6.2 Proxy Code Template . . . 11

6.3 URL Rewriting (Critical) . . . 11

6.4 Handling All Content Types . . . 11

7. Troubleshooting Guide 13

8. Quick Reference Card 14

8.1 Decision Tree: What Type of Site Is This? . . . 14

8.2 Tool Selection Guide . . . 14

8.3 Common Header Requirements . . . 14

9. How to Verify Your Streams Work **15**

9.1 Quick Check with curl 15

9.2 Full Check with ffprobe 15

9.3 Play Test with ffmpeg 15

9.4 Segment Verification (for encrypted streams) 15

10. Complete Workflow Example **16**

1. The Big Picture

Every streaming website shows you a video player in your browser. Behind that player, there is a chain of steps that turns a movie title into a playable video URL. This guide teaches you how to trace that chain and replicate it in Node.js.

Think of it like a treasure hunt. The movie title is the starting clue. Each step gives you the next clue. The treasure at the end is a direct URL to a .m3u8 (HLS) or .mp4 or .mkv file that any video player can play.

1.1 The Universal Chain

Every streaming site follows this pattern:

```
Step 1: Start with a movie title or ID (like a TMDB ID)

Step 2: Find the movie page on the streaming site

Step 3: Extract stream URLs from that page (or from a hidden API)

Step 4: Decrypt or decode the URLs if they are encrypted

Step 5: Return the final direct playable URL
```

> TIP: Some sites have simple chains (just 2 steps). Others have complex chains (5+ steps with encryption). This guide covers both.

1.2 What You Need

Only 3 tools:

```
1. Node.js (v18+) — runs JavaScript code, has built-in HTTPS, crypto, WebAssembly

2. curl — a command-line HTTP client (already installed on most systems)

3. ffprobe/ffmpeg — for testing if streams actually play (optional but recommended)
```

> That is it. No browser. No Playwright. No npm install.

2. Step 1: Reconnaissance (Scout the Site)

Before you can extract streams, you need to understand how the site works. This means fetching the homepage and a movie page, then looking for clues.

2.1 Fetch the Homepage

Use curl to download the homepage HTML:

```
curl -sL -H 'User-Agent: Mozilla/5.0' 'https://streaming-site.com/' > home.html
```

2.2 Fetch a Movie Page

Find any movie link on the homepage, then fetch that page too:

```
curl -sL -H 'User-Agent: Mozilla/5.0' 'https://streaming-site.com/movie/299534' > movie.html
```

2.3 Look for Clues

Search the downloaded HTML for these patterns:

What you find	What it means	Next step
.m3u8 or .mp4 URL	Direct stream URL found!	Done — play it directly
/api/ endpoint	Site has an API	Call the API to get streams
<iframelt;iframe src="...">	Embed from another site	Fetch the iframe page
download links	File host mirrors	Follow each mirror link
base64 string (?id=...)	Encoded URL	Decode: Buffer.from(id, 'base64')
No stream URLs in HTML	JavaScript SPA	Download JS bundles (Ch 3)
Encrypted data	API encrypts responses	Find decryption key (Ch 5)

> TIP: If the HTML has no stream URLs, the site is a JavaScript SPA (Single Page App). The stream URLs are loaded dynamically by JavaScript. See Chapter 3.

3. Step 2: Find the Hidden API

If the stream URLs are not in the HTML, they are loaded by JavaScript. You need to find which JavaScript file makes the API call that returns stream URLs.

3.1 Download JavaScript Bundles

Find all <script src="..."> tags in the HTML and download them:

```
# Find script tags
grep -o 'src="[^"]*\.js"' movie.html

# Download each one
curl -sL 'https://site.com/assets/index-ABC123.js' > main.js
```

3.2 Search for API URLs

Search the JavaScript for patterns that look like API calls:

```
# Look for fetch() calls
grep -o 'fetch(`[^`]*`' main.js

# Look for URLs with template variables (${})
grep -oE 'https?://[^"\' ]+\$\{[^}]+\}[^"\' ]*' main.js

# Look for /api/ paths
grep -oE '/api/[a-z0-9/_\-]+' main.js

# Look for baseURL
grep -o 'baseURL["\' ]*[:=]["\' ]*[^"\' ]+' main.js
```

3.3 Understanding Webpack Chunks

Next.js and React sites split their JavaScript into multiple "chunks" (small files). The main bundle loads other chunks dynamically. You need to find which chunk has the stream code.

The chunk ID to filename mapping is in the webpack runtime JS:

```
# The webpack runtime contains a function like:
# p.u = function(e) { return 6476===e ? "chunks/6476.abc123.js" : ... }

# Extract chunk IDs and their hashes:
grep -oE '[0-9]+:"[a-f0-9]+"' webpack.js
# Output: 6476:"1364eecee369c803", 9230:"719542700d7b5fe3", ...

# Then download the chunk:
curl -sL 'https://site.com/_next/static/chunks/9230.719542700d7b5fe3.js' > player.js
```

> TIP: Search each downloaded chunk for keywords like "m3u8", "mp4", "sources", "stream", "fetch", "url", "player" to find the one with stream logic.

4. Step 3: Bypass Cloudflare

Many streaming sites use Cloudflare (CF) to protect against bots. CF has several protection layers. Here is how to bypass each one without a browser.

4.1 Protection Type 1: TLS Fingerprinting (JA3)

Problem: CF checks the TLS handshake fingerprint. Node.js has a unique fingerprint that CF blocks with HTTP 403.

Solution: Use curl instead of Node.js HTTPS. Curl's TLS fingerprint looks like a real browser and is accepted by CF.

```
const { execFileSync } = require('child_process');

function fetchWithCurl(url) {
  return execFileSync('curl', ['-sL', '--max-time', '30', url],
    { encoding: 'buffer', maxBuffer: 50*1024*1024 });
}
```

4.2 Protection Type 2: User-Agent Detection

Problem: Some CF sites reject the full Chrome User-Agent string as a bot.

Solution: Use a shorter, simpler User-Agent that passes CF bot detection:

```
// REJECTED by some CF sites (403 Forbidden):
'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/131.0.0.0 Safari/537.36'

// ACCEPTED (200 OK):
'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36'
```

4.3 Protection Type 3: Referer Gating

Problem: Some CDNs only serve streams if the Referer header matches their domain. Without the correct Referer, you get 403.

Solution: Include the Referer header in your request and pass it to the player:

```
// When fetching the stream, include Referer:
const response = await fetch(streamUrl, {
  headers: { 'Referer': 'https://player.videasy.to/' }
});

// When returning the stream, tell the player to use this Referer:
{
  url: streamUrl,
  behaviorHints: {
    proxyHeaders: { request: { Referer: 'https://player.videasy.to/' } }
  }
}
```

4.4 Quick Decision Tree

```
IF you get HTTP 403 from a site:
  TRY 1: Use curl instead of Node.js HTTPS (bypasses JA3)
  TRY 2: Use a shorter User-Agent string
  TRY 3: Add a Referer header matching the site's domain
  TRY 4: Check if the site has a separate API subdomain without CF

IF none of these work, the site uses a full CF challenge page.
In that case, look for an alternative API endpoint that does NOT go through CF.
```

5. Step 4: Decrypt Encrypted Streams

Some sites encrypt their stream URLs or segment data. This chapter covers every encryption type I have encountered and how to decrypt each one in pure Node.js.

5.1 Base64-Encoded URLs (Easy)

Some sites hide URLs by encoding them as base64:

```
// URL: /generate.php?id=aHR0cHM6Ly9jZG4uZXhhbXBsZS5jb20vdmkgZG8ubTNO

// Decode it:
const decoded = Buffer.from(base64String, 'base64').toString('utf8');
// Result: "https://cdn.example.com/video.m3u8"
```

5.2 XOR-Encrypted Segments (Medium)

Some CDNs disguise video segments as image files (.webp or .png). The file starts with a fake image header, then the rest is XOR-encrypted MPEG-TS data. The XOR key is hardcoded in the site's JavaScript.

How to find the key:

```
1. Download the site's hls.js or player JavaScript file
2. Search for a 16-byte array of numbers near the words "webp" or "png"
3. It looks like: [157, 42, 241, 71, 179, 142, 92, 112, ...]
4. That is your XOR key
```

How to decrypt:

```
const KEY = Buffer.from([157, 42, 241, 71, 179, 142, 92, 112, 166, 25, 228, 59, 216, 98, 15,
197]);

// Check if it is a fake WebP (starts with "RIFF....WEBP")
if (body[0] === 0x52 && body[8] === 0x57 && body[11] === 0x50) {
  const payload = body.slice(12); // Remove 12-byte fake header
  const decrypted = Buffer.alloc(payload.length);
  for (let i = 0; i < payload.length; i++) {
    decrypted[i] = payload[i] ^ KEY[i % 16]; // XOR each byte
  }
  // Result: valid MPEG-TS (first byte should be 0x47)
}
```

> TIP: To verify decryption worked, check that the first byte is 0x47 (MPEG-TS sync byte) and that every 188th byte is also 0x47.

5.3 WASM Decryption (Hard)

Some sites embed a WebAssembly (WASM) module in the page as base64. This module contains functions that derive decryption keys. Node.js can run WASM natively — no browser needed.

Step-by-step:

```
// Step 1: Find the base64 WASM in the page HTML
const wasmB64 = html.match(/w_payload:"([^"]+)"/)[1];

// Step 2: Compile and instantiate in Node.js
const wasmBytes = Buffer.from(wasmB64, 'base64');
const mod = await WebAssembly.compile(wasmBytes);
const inst = await WebAssembly.instantiate(mod, {});

// Step 3: Read the player JS to find which WASM functions to call
// Look for: ex._s(), ex._r(), ex._c() in the player code
// These are the WASM exports for: set_seed, run_derivation, get_key
```

```
// Step 4: Set up WASM memory (write inputs, call functions, read outputs)
const mem = new Uint8Array(inst.exports.memory.buffer);
mem.set(inputBuffer, 1000); // Write input at offset 1000
inst.exports._s(seedValue); // Set the seed
inst.exports._r(1000, ...); // Run derivation
const key = Buffer.from(mem.subarray(outputOffset, outputOffset + 32));

// Step 5: Use the derived key for AES decryption (see 5.4)
```

> WARN: The exact memory layout and function arguments depend on the site. You must read the player JavaScript to find how it calls the WASM functions. Look for patterns like: ex._s(v), ex._r(I, P, U, nt, k).

5.4 AES-256-CBC Decryption (Hard)

After WASM derives a key fragment, the final URL is AES-256-CBC encrypted. The full key derivation chain: WASM output -> PBKDF2 -> XOR with seed -> SHA-256 -> AES key.

```
const crypto = require('crypto');

// Step 1: PBKDF2 (derive 32 bytes from WASM output + seed)
const W = crypto.pbkdf2Sync(wasmKey, seed, 1000, 32, 'sha256');

// Step 2: XOR with seed bytes
const tt = Buffer.from(W);
for (let i = 0; i < 32; i++) tt[i] ^= seed.charCodeAt(i % seed.length);

// Step 3: SHA-256 to get final AES key
const aesKey = crypto.createHash('sha256').update(tt).digest();

// Step 4: AES-256-CBC decrypt
const decipher = crypto.createDecipheriv('aes-256-cbc', aesKey, iv);
const decrypted = Buffer.concat([decipher.update(ct), decipher.final()]);
// Result: the stream URL as a UTF-8 string
```

5.5 SHA-256 Obfuscated Field Names

Some sites do not use fixed JSON key names. Instead, the key names are derived from a per-request seed by chaining SHA-256 hashes. This makes it impossible to hardcode field names — you must compute them dynamically.

```
function sha256(s) {
  return crypto.createHash('sha256').update(s, 'utf8').digest('hex');
}

// Chain: hash the seed 3 times, then hash the result 3 more times
let e = seed;
for (let i = 0; i < 3; i++) e = sha256(e + i.toString());
let a = e;
for (let i = 0; i < 3; i++) a = sha256(a + i.toString());

// The hash determines field names:
// containerName = "cd_" + e.substring(24, 32)
// keyField = "kf_" + e.substring(8, 16)
// tokenField = e.substring(48, 64) + "_" + e.substring(56, 64)

// Search the page HTML for these dynamic field names to extract values
```

5.6 Custom Stream Ciphers (Hardest)

Some sites build their own encryption from scratch, combining multiple algorithms: RC4 key schedule + FNV-1a hash + xmur3 finalizer + custom PRNG. These are harder to port because the

PRNG often uses the output buffer itself as its internal state (reading from and writing to the same array).

How to approach this:

```
1. Extract the crypto functions from the site's JavaScript chunk

2. Identify the constants (usually arrays of integers at the top)

3. Identify the helper functions: hash, rotate-left, RC4 key schedule

4. Identify the PRNG: does it read from the output buffer? If yes, you must
   use the SAME buffer for both PRNG state and keystream output

5. Look for a magic header (like "mvm1" = bytes [109, 118, 109, 49]) that
   verifies decryption was successful

6. Port each function to Node.js, then chain them together

7. Test by checking if the magic header appears at the start of the output
```

> WARN: If porting fails after multiple attempts, you can still use Playwright as a fallback to run the decryption in the browser context. But try Node.js first — it is always faster.

6. Building a Decryption Proxy

When streams are encrypted, standard video players (VLC, mpv, Stremio) cannot play them directly. The solution is to build a small local HTTP server (proxy) that fetches the encrypted content, decrypts it, and serves it as plain HLS/MPEG-TS.

6.1 How the Proxy Works

```
Player (VLC) --> Local Proxy (port 7654) --> CDN (encrypted)
                                    |
                                    decrypt here
                                    serve as plain TS
```

6.2 Proxy Code Template

```
const http = require('http');

http.createServer(async (req, res) => {
  const url = new URL(req.url, 'http://localhost');
  const targetUrl = url.searchParams.get('url');

  // 1. Fetch the encrypted content from the CDN
  const upstream = await fetch(targetUrl, {
    headers: { 'Referer': 'https://target-cdn.com/' }
  });
  const body = Buffer.from(await upstream.arrayBuffer());

  // 2. Detect what type of content it is
  if (isWebP(body)) {
    // XOR decrypt the segment
    const decrypted = xorDecrypt(body.slice(12), KEY);
    res.writeHead(200, { 'Content-Type': 'video/mp2t' });
    res.end(decrypted);
  } else if (isBase64M3u8(body)) {
    // Base64 decode + XOR decrypt the playlist
    const decoded = Buffer.from(body.toString(), 'base64');
    const decrypted = xorDecrypt(decoded, sessionKey);
    // Rewrite URLs to route through this proxy
    const rewritten = rewriteUrls(decrypted.toString(), port);
    res.writeHead(200, { 'Content-Type': 'application/vnd.apple.mpegurl' });
    res.end(rewritten);
  } else {
    // Plain content — pass through
    res.writeHead(200);
    res.end(body);
  }
}).listen(7654);
```

6.3 URL Rewriting (Critical)

The m3u8 playlist contains URLs for segments. If those URLs are encrypted or need special headers, you must rewrite them to go through your proxy.

```
// Original: https://cdn.example.com/seg-0-f1-v1-a0.webp
// Rewritten: http://localhost:7654/seg.ts?url=&e;=.ts

// The "/seg.ts" path and "&e;=.ts" suffix trick ffmpeg into accepting
// the URL as a .ts file (otherwise it rejects .webp segment extensions)

// For sub-playlists (video.m3u8, audio.m3u8):
// http://localhost:7654/playlist.m3u8?url=&key;=
```

6.4 Handling All Content Types

The proxy must handle: 1) WebP disguised segments, 2) PNG disguised segments, 3) Base64-encoded m3u8 playlists, 4) Plain m3u8 playlists (already decoded), 5) Plain segments (pass through). Detect by checking magic bytes at the start of the response body.

7. Troubleshooting Guide

If something is not working, follow this decision tree to find and fix the problem.

Problem	Likely Cause	Fix
HTTP 403 from site	Cloudflare blocking	Use curl instead of Node.js HTTPS
HTTP 403 from CDN	Missing Referer header	Add Referer matching the player domain
Empty response	Rate limited	Add delay between requests (500ms+)
JSON parse error	Response is encrypted	Check if response is base64 or XOR
WASM instantiation fails	Wrong WASM bytes	Check base64 decoding is correct
AES decrypt throws error	Wrong key or IV	Verify PBKDF2 + XOR + SHA-256 chain
Decrypted data is garbage	Wrong XOR key	Search JS for hardcoded byte array
m3u8 is encrypted text	Base64 + XOR encryption	Base64 decode, then XOR with session key
Segments are .webp files	Disguised MPEG-TS	Strip 12-byte header, XOR with 16-byte key
ffmpeg rejects segment URLs	.ts extension mismatch	Use /seg.ts?url=...&e=.ts proxy URL
No streams found	Site changed API	Re-download JS bundles, search for new URLs
Stream URL returns 404	Token expired	Re-fetch fresh token from API
VLC shows black screen	Missing CORS headers	Add Access-Control-Allow-Origin: * to proxy
Proxy is slow	Synchronous XOR loop	Use Buffer.alloc + for loop (faster than spread)

8. Quick Reference Card

8.1 Decision Tree: What Type of Site Is This?

```
IF HTML contains .m3u8 or .mp4 URLs:
  -> Simple site. Extract URLs directly. Done.

IF HTML contains /api/ endpoints:
  -> API site. Call the API, parse JSON response. Done.

IF HTML has <iframe> with embed URLs:
  -> Embed site. Fetch the iframe page, repeat from top.

IF HTML has download links (mirror sites):
  -> Download site. Follow each link to find direct file URL.

IF HTML has no stream URLs and loads JS bundles:
  -> SPA site. Download JS, search for API calls (Ch 3).

IF JS bundle contains encryption code (AES/XOR/WASM):
  -> Encrypted site. Find keys, decrypt responses (Ch 5).

IF decrypted response contains encrypted m3u8 + segments:
  -> Fully encrypted. Build a decryption proxy (Ch 6).
```

8.2 Tool Selection Guide

Task	Tool	Why
Fetch HTML from CF site	curl	Bypasses JA3 fingerprint
Fetch HTML from non-CF site	Node.js https	Faster, no subprocess
Fetch TMDB info	Node.js https	TMDB has no CF
Decode base64	Buffer.from(b64, 'base64')	Built-in Node.js
XOR decrypt	Buffer + for loop	Simple and fast
AES decrypt	crypto.createDecipheriv	Built-in Node.js
SHA-256 hash	crypto.createHash	Built-in Node.js
PBKDF2 key derivation	crypto.pbkdf2Sync	Built-in Node.js
WASM execution	WebAssembly.compile + instantiate	Built-in Node.js
Local HTTP proxy	http.createServer	Built-in Node.js
Verify stream plays	ffprobe / ffmpeg	External CLI tool
Search JS for patterns	grep or regex	Find API URLs and keys

8.3 Common Header Requirements

CDN / Site	Required Header	Value
peakstorm.top (StreamXTV)	Referer	https://player.videasy.to/
img1.*.com (Citadel/Rivestream)	Referer	https://rivestream.ru/
fetch8.flixcloud.cc (ReAnime)	Referer	https://flixcloud.cc/
vault-95.atomic4cdn.top (ReAnime)	Referer	https://flixcloud.cc/
proxy.valhallastream.dpdns.org	Referer	https://rivestream.ru/
pixeldrain.com API	(none)	CORS enabled, any UA works
pub-*.r2.dev (KMMovies)	(none)	No auth needed
moon.peakstorm.top (Videasy)	Referer	https://player.videasy.to/

9. How to Verify Your Streams Work

After extracting stream URLs, always verify they actually play. This saves you from shipping broken providers.

9.1 Quick Check with curl

```
# Check if the URL returns valid HLS:
curl -sL -H 'Referer: https://player.videasy.to/' '' | head -1

# If output starts with "#EXTM3U" -> valid HLS playlist
# If output starts with binary -> encrypted segment (needs decryption)
```

9.2 Full Check with ffprobe

```
ffprobe -v error -show_streams \
  -headers $'Referer: https://player.videasy.to/\r\n' \
  ''

# Look for:
# codec_name=h264 (video codec found)
# width=1920, height=1080 (resolution matches)
# codec_name=aac (audio codec found)
```

9.3 Play Test with ffmpeg

```
# Extract 5 seconds of video to verify end-to-end playback:
ffmpeg -y \
  -protocol_whitelist "file,http,https,tcp,tls,crypto,data" \
  -allowed_extensions ALL \
  -i "" -t 5 -c copy output.mp4

# If output.mp4 exists and is > 0 bytes -> stream is playable!
```

9.4 Segment Verification (for encrypted streams)

After decrypting a segment, verify it is valid MPEG-TS by checking sync bytes:

```
// MPEG-TS packets are 188 bytes, each starting with 0x47
let valid = true;
for (let i = 0; i < decrypted.length; i += 188) {
  if (decrypted[i] !== 0x47) { valid = false; break; }
}
console.log(valid ? 'Valid MPEG-TS' : 'Decryption failed');
```

10. Complete Workflow Example

Here is the complete workflow from start to finish, using a hypothetical encrypted streaming site as an example.

#	Action	Tool
1	Fetch the homepage HTML	curl
2	Search for stream URLs in HTML	grep / regex
3	If no URLs found, download JS bundles	curl
4	Search JS for API endpoints and encryption	grep / regex
5	If encrypted, find keys in JS (XOR array, WASM, base64)	grep
6	Call the API to get stream data	Node.js https or curl
7	If response is base64, decode it	Buffer.from(b64)
8	If response is XOR encrypted, decrypt it	Buffer XOR loop
9	If WASM is needed, compile and run it	WebAssembly.compile
10	If AES is needed, derive key and decrypt	crypto module
11	Extract final stream URL from decrypted data	JSON.parse
12	If segments are encrypted, build a proxy	http.createServer
13	Verify the stream plays with ffprobe	ffprobe CLI
14	Package as a provider module	module.exports

This guide covers the complete methodology for extracting direct playable streams from any web source. The techniques are universal — they work on movies, TV shows, anime, and any content type. The only tools needed are Node.js and curl.

Key principle: The browser is just a JavaScript runtime that makes HTTP requests. Replicate what the browser does, in pure Node.js, and you never need a browser.